université
PARIS-SACLAY

PERFORMANCE BENCHMARKING



FOR COVID-19

DATA ANALYTICS

Using Docker & Studio3T

Author:
Akshay KHOKAN

Supervised by :
Nazim AGOULMINE

M2 GENIOMHE-AI

Table of contents

ABSTRACT **3**

FRAMEWORKS SETUP **4**

1. Hardware and Software Environment *4*

2. Launch of Hadoop, PySpark and MongoDB *4*

PRIOR ANALYSIS **6**

1. Content of the dataset *6*

2. Proposed extracted information *7*

ANALYSIS & BENCHMARK **9**

1. MapReduce approaches for each question *9*

2. Analyses with different frameworks *9*

2.1. Relationship between weekly COVID-19 cases and deaths *9*

2.2. Association between population size and COVID-19 cases and deaths *18*

2.3. Differences in cumulative deaths-to-cases ratios across countries *26*

2.4. Continental differences in reported COVID-19 cases and deaths *33*

2.5. Identification of peak COVID-19 weeks across continents *39*

CONCLUSION **46**

REFERENCES **47**

Table of figures

TABLES

Table 1. Used machine specifications....4
Table 2. COVID-19 dataset description....7
Table 3. Pearson's correlation between numerical variables....8
Table 4. Corresponding MapReduce approaches for Research questions....9

FIGURES

Figure 1. Execution time comparison of Hadoop, MongoDB, and PySpark....16
Figure 2. Pearson correlation coefficients between weekly cases and deaths across countries....17
Figure 3. Execution time comparison of Hadoop, MongoDB, and PySpark....24
Figure 4. Relationship between country population size and cumulative cases and deaths....25
Figure 5. Execution time comparison of Hadoop, MongoDB, and PySpark....31
Figure 6. Countries with the 10 lowest and 10 highest cumulative deaths-to-cases ratios. 32
Figure 7. CPU and memory usage of the frameworks before and during the execution....37
Figure 8. Cumulative reported COVID-19 cases and deaths across continents....38
Figure 9. CPU and memory usage of the frameworks before and during the execution....44
Figure 10. Weekly peaks of COVID-19 cases and deaths across continents....45

Performance Benchmarking MongoDB, Hadoop & PySpark for COVID-19 Data Analytics.

Akshay KHOKAN [1]

[1] Master's Student (Year 2) in GENIOMHE-AI, Paris-Saclay University

ABSTRACT

This project investigates the use and performance of three Big Data frameworks: Hadoop, MongoDB, and PySpark, for analysing epidemiological data. Using a weekly COVID-19 dataset, five analytical studies were conducted to explore the relationship between reported cases and deaths, the influence of population size, differences between countries and continents, and the timing of epidemic peaks. The same MapReduce operations were implemented across the three frameworks to compare their outputs, execution times, and resource consumption. Some of these studies implicated the statistical Pearson's correlation test. All three frameworks produced identical analytical results, although their performance differed. MongoDB recorded the shortest execution time, followed by PySpark and Hadoop. CPU and memory monitoring also revealed differences in resource usage, with Hadoop and PySpark showing higher CPU consumption during execution. These findings highlight the different characteristics of each framework and show that performance depends on factors such as data access, initialization overhead, hardware configuration, and workload complexity. This project demonstrates how Big Data technologies can support epidemiological data analysis while highlighting the importance of considering both analytical consistency and computational performance. CPU and memory usages were considered only for the fourth and fifth studies.

Keywords: Big Data, Hadoop, MongoDB, PySpark, MapReduce, COVID-19, performance benchmarking.

FRAMEWORKS SETUP

Before benchmarking and analysing any kind of data, a prior installation of the framework is needed (here in Docker).

1. Hardware and Software Environment

Three different frameworks are benchmarked in the same machine, thus it is important to show the machine's specifications to avoid bias and to enable reproducibility. (***Table 1***)

Component	Specification
CPU	AMD Ryzen 7 5700U (8 cores, 16 threads)
RAM	14 GiB (6.8 GiB available)
Storage	476.95 GiB Kingston NVMe SSD

Table 1. Used machine specifications.

The following command line will be used to check all the CPU and memory usage by the frameworks.

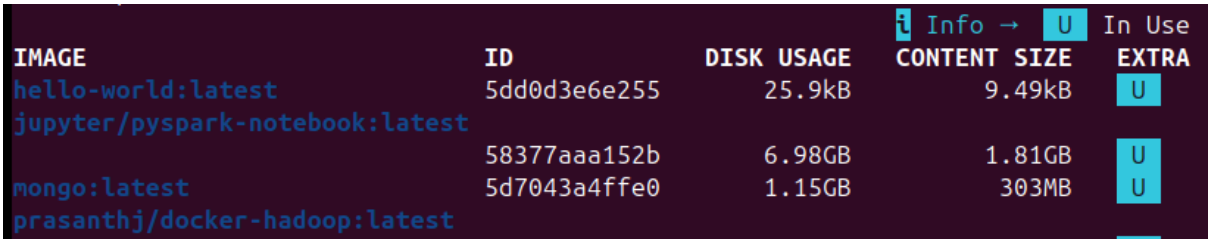
```
docker stats
```

And this one will be used to check the execution for the Hadoop container.

```
time bash -c
```

2. Launch of Hadoop, PySpark and MongoDB

All respective images of Hadoop, PySpark and MongoDB are in Docker as shown below.



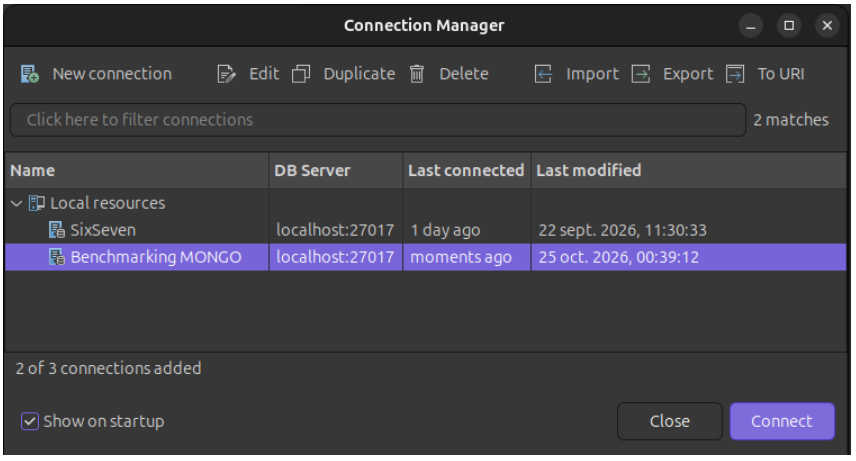
Each container is run with Docker, however MongoDB is exclusively used on Studio3T, to allow a better visualization of the data.

```
docker run -d \
  --name mongo-benchmark \
  -p 27017:27017 \
  -v mongo_data:/data/db \
  mongo:latest

docker run -d \
  --name pyspark-benchmark \
  -p 8888:8888 \
  -p 4040:4040 \
  -v "$PWD:/home/jovyan/work" \
  jupyter/pyspark-notebook:latest

docker run -d \
  --name hadoop \
  -p 9870:9870 \
  -p 8088:8088 \
  -p 9000:9000 \
  -v "$PWD:/work" \
  prasanthj/docker-hadoop
```

The connection between Studio3T and the MongoDB container is now established.



PRIOR ANALYSIS

The frameworks are ready to be used on any kind of data. This section covers the description of the dataset used for the benchmark and what information can be extracted and shown.

1. Content of the dataset

The COVID-19 dataset contains **10,647 observations and 10 variables** (***Table 2***), covering **215 countries and territories across six continents.** It includes **weekly case and death counts, population data, dates, and COVID-19 notification rates, with 58 distinct weeks recorded**. The dataset has some quality issues : missing values affect the country codes, population data, and notification rates, while 10 observations have negative case counts and two have negative death counts, possibly due to data corrections. These issues should be considered when preparing the data and interpreting the benchmark. This information was obtained after parsing the dataset in R.

```
> # Size of dataset
> dim(covid)
[1] 10647    10

> # Names of columns and stats
> str(covid)
...   #

> # Range of dates and number of weeks
> range(covid$date)
[1] "01/02/2021" "31/08/2020"
> n_distinct(covid$year_week)
[1] 58

> # Countries and continent
> n_distinct(covid$countriesAndTerritories)
[1] 215
> table(covid$continentExp)
 Africa America    Asia  Europe Oceania   Other
   2607    2374    2249    2945     463       9

> # Quality of data : can lead to eventual problems for benchmark...
> colSums(is.na(covid))
> sum(covid$cases_weekly < 0)
[1] 10
> sum(covid$deaths_weekly < 0)
[1] 2
```

Column	Description
dateRep	Date associated with the reported data.
year_week	Year and week corresponding to the observation.
cases_weekly	Number of new COVID-19 cases reported during the week.
deaths_weekly	Number of new COVID-19 deaths reported during the week.
countriesAndTerritories	Name of the territory associated with the observation.
geoId	Two-letter geographical identifier for the territory.
countryterritoryCode	Three-letter territory code.
popData2019	Estimated population of the territory in 2019.
continentExp	Continent assigned to the territory.
notification_rate_per_100000_population_14.days	COVID-19 notification rate over 14 days per 100,000 inhabitants, used to compare the spread of the virus across populations of different sizes.

Table 2. COVID-19 dataset description.
This figure was generated using the GT library in R.

2. Proposed extracted information

Instead of manually choosing the variables to study the dataset, Pearson's correlation (2 by 2) approach was chosen to pick the information that is the most correlated. PCA was initially intended to be used here, but given the dimension of the dataset, the Pearson approach is better. This analysis led to 2 initial questions, depending on the strength of the correlations. (***Table 3***)

- *How does the relationship between weekly COVID-19 cases and deaths vary across countries?*

- *To what extent are population size and COVID-19 case and death counts associated across countries?*

Variable	Cases weekly	Deaths weekly	Population	14-day rate
Cases weekly	1.000	0.850	0.266	0.242
Deaths weekly	0.850	1.000	0.258	0.216
Population	0.266	0.258	1.000	−0.048
14-day rate	0.242	0.216	−0.048	1.000

Table 3. Pearson's correlation between numerical variables.
14-day rate is the variable corresponding to "notification_rate_per_100000_population_14.days". Qualitative variables were excluded from the analysis. This figure was generated using the GT library in R.

To extend the analysis beyond numerical correlations, the dataset's qualitative variables were then considered, particularly continent, country, and week. This led to 3 additional research questions:

- *How does the ratio of reported deaths to reported cases vary across countries and continents?*

The country and continent variables enable geographical comparisons of reported deaths relative to reported cases.

- *How did the total numbers of reported COVID-19 cases and deaths differ across continents?*

Grouping weekly cases and death counts by continent makes it possible to compare the strength of their relationship across geographical regions.

- *During which weeks did each continent record its highest numbers of weekly COVID-19 cases and deaths?*

Grouping weekly case and death counts by continent makes it possible to identify and compare the timing of reported epidemic peaks across regions.

These five questions will guide the MapReduce analyses. They involve several operations, such as grouping, calculating sums and averages, calculating ratios and finding peaks. These tasks are useful to explore the dataset and compare how Hadoop, MongoDB, and PySpark process the same data.

ANALYSIS & BENCHMARK

Five different questions were found after the data exploration. This section covers the comparison of the frameworks by applying equivalent analytical approaches to the dataset and by evaluating their execution time and ability to produce consistent results.

1. MapReduce approaches for each question

For each question, the Map method organizes the data by relevant variables, while the Reduce one aggregates the results and performs the required calculations to identify trends and relationships in the COVID-19 dataset. (***Table 4***)

No.	Research Question	MapReduce Approach
1	Relationship between weekly cases and deaths across countries	**Map**: Associate each country and week with its reported case and death counts. **Reduce**: Group observations by country and calculate the Pearson correlation between weekly cases and deaths for each country.
2	Association between population size and COVID-19 case and death counts	**Map**: Associate each country with its population and reported case and death counts. **Reduce**: Aggregate case and death counts by country, retain one population value per country, and calculate the associations between population size and total cases and deaths.
3	Variation in the deaths-to-cases ratio	**Map**: Associate each country and continent with its reported case and death counts. **Reduce**: Aggregate case and death counts by country and continent, then calculate the ratio of total reported deaths to total reported cases for each group.
4	Differences in reported cases and deaths across continents	**Map**: Associate each continent with the reported case and death counts from each observation. **Reduce**: Aggregate reported cases and deaths by continent to compare the total numbers across geographical regions.
5	Epidemic peaks by continent	**Map**: Associate each continent and week with its reported case and death counts. **Reduce**: Aggregate weekly case and death counts by continent, then identify the week or weeks with the highest values for cases and deaths in each continent.

Table 4. Corresponding MapReduce approaches for Research questions.
Overview of the approaches selected for each question, describing how the data are processed to investigate COVID-19 trends and relationships. This figure was generated using the GT library in R.

2. Analyses with different frameworks

2.1. Relationship between weekly COVID-19 cases and deaths

Research question: *How does the relationship between weekly reported COVID-19 cases and deaths vary across countries?*

The objective is to measure the relationship between weekly COVID-19 cases and deaths in each country. The same MapReduce logic is implemented in **Hadoop**, **MongoDB** (through Studio 3T), and **PySpark** to compare their outputs and performance.

The Map phase processes each observation and emits the country identifier along with the weekly case and death counts and the sufficient statistics required to calculate Pearson's correlation coefficient (n, x, y, x^2, y^2, and xy). During the Shuffle and Sort phase, Hadoop groups all observations by country. The Reduce phase aggregates these statistics for each country and calculates the Pearson correlation coefficient, producing one final result per country with the number of observations and the corresponding correlation coefficient.

a) Hadoop approach

Here are the two corresponding Python scripts used for the MapReduce approach in Hadoop.

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
import sys
import csv

reader = csv.reader(sys.stdin)
for row in reader:
    try:
        # Ignore the header
        if not row or row[0] == "dateRep":
            continue

        # Dataset columns:
        # 2 = cases_weekly
        # 3 = deaths_weekly
        # 4 = countriesAndTerritories
        cases = row[2].strip()
        deaths = row[3].strip()
        country = row[4].strip()

        if not country or not cases or not deaths:
            continue

        x = float(cases)
        y = float(deaths)

        # n, x, y, x², y², xy
        print("{}\t1\t{}\t{}\t{}\t{}\t{}".format(
            country, x, y, x * x, y * y, x * y))

    except (ValueError, IndexError):
        continue
```

Mapper Python file used for this specific Hadoop approach.

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-

import sys
import math

def emit_result(country, n, sum_x, sum_y, sum_x2, sum_y2, sum_xy):
    if country is None:
        return

    numerator = n * sum_xy - sum_x * sum_y
    variance_x = n * sum_x2 - sum_x * sum_x
    variance_y = n * sum_y2 - sum_y * sum_y

    if n < 2 or variance_x <= 0 or variance_y <= 0:
        correlation = "NA"
    else:
        denominator = math.sqrt(variance_x * variance_y)
        correlation = numerator / denominator

    print("{}\t{}\t{}".format(country, n, correlation))

current_country = None
n = 0
sum_x = 0.0
sum_y = 0.0
sum_x2 = 0.0
sum_y2 = 0.0
sum_xy = 0.0

for line in sys.stdin:
    parts = line.strip().split("\t")
    if len(parts) != 7:
        continue
    country = parts[0]
    try:
        count = int(parts[1])
        x = float(parts[2])
        y = float(parts[3])
        x2 = float(parts[4])
        y2 = float(parts[5])
        xy = float(parts[6])
    except ValueError:
        continue

    if current_country is not None and country != current_country:
        emit_result(
            current_country, n, sum_x, sum_y,
            sum_x2, sum_y2, sum_xy)
```

```
        n = 0
        sum_x = 0.0
        sum_y = 0.0
        sum_x2 = 0.0
        sum_y2 = 0.0
        sum_xy = 0.0

    current_country = country

    n += count
    sum_x += x
    sum_y += y
    sum_x2 += x2
    sum_y2 += y2
    sum_xy += xy

emit_result(
    current_country, n, sum_x, sum_y,
    sum_x2, sum_y2, sum_xy
)
```

Reducer Python file used for this specific Hadoop approach.

An output is obtained using these files with Hadoop approach, here are its 5 first and last lines.

```
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat COVID/MR1/part-* |
head -n 5
Afghanistan    58    0.59834172784
Albania    49    0.904116305346
Algeria    58    0.721276492119
Andorra    48    0.306637940193
Angola    47    0.800339489828

root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat COVID/MR1/part-* |
tail -n 5
Wallis_and_Futuna    17    NA
Western_Sahara    42    -0.0029736692075
Yemen    44    0.926860976688
Zambia    47    0.913163306103
Zimbabwe    47    0.809748066005
```

b) MongoDB approach

The following script is executed in Studio3T, with the MapReduce approach.

```
var mapper = function() {
   var country = this.countriesAndTerritories;
   var x = this.cases_weekly;
   var y = this.deaths_weekly;
   if (
       typeof country !== "string" ||
       typeof x !== "number" ||
       typeof y !== "number"
   ) {
       return;
   }
   emit(country, {
       n: 1,
       sum_x: x,
       sum_y: y,
       sum_x2: x * x,
       sum_y2: y * y,
       sum_xy: x * y
   });
};
var reducer = function(key, values) {
   var result = {
       n: 0,
       sum_x: 0,
       sum_y: 0,
       sum_x2: 0,
       sum_y2: 0,
       sum_xy: 0
   };
   values.forEach(function(v) {
       result.n += v.n;
       result.sum_x += v.sum_x;
       result.sum_y += v.sum_y;
       result.sum_x2 += v.sum_x2;
       result.sum_y2 += v.sum_y2;
       result.sum_xy += v.sum_xy;
   });
   return result;
};
var finalize = function(key, value) {
   var n = value.n;
   var variance_x = n * value.sum_x2 -
                    value.sum_x * value.sum_x;
   var variance_y = n * value.sum_y2 -
                    value.sum_y * value.sum_y;
```

```
    if (n < 2 || variance_x <= 0 || variance_y <= 0) {
        value.pearson_r = null;
    } else {
        var numerator = n * value.sum_xy - value.sum_x * value.sum_y;
        var denominator = Math.sqrt(variance_x * variance_y);
        value.pearson_r = numerator / denominator;
    }
    return value;
};
db.covid_weekly.mapReduce(
    mapper,
    reducer,
    {
        out: { replace: "MR1_MongoDB" },
        finalize: finalize
    }
);
```

Studio3T file used for this specific MongoDB approach.

An output is obtained using these files with Mongo approach, here are its 5 first and last lines.

MR1_MongoDB > value > n

{Document id}	n	pearson_r
Afghanistan	58.0	0.5983417278404136
Albania	49.0	0.9041163053459342
Algeria	58.0	0.7212764921189101
Andorra	48.0	0.3066379401933389
Angola	47.0	0.8003394898282855

5 first lines of the MongoDB output.

MR1_MongoDB > value > {Document id}

{Document id}	n	pearson_r
Wallis_and_Futuna	17.0	null
Western_Sahara	42.0	-0.002973669207501232
Yemen	44.0	0.9268609766875067
Zambia	47.0	0.9131633061030852
Zimbabwe	47.0	0.8097480660046228

5 last lines of the MongoDB output.

c) PySpark approach

For this approach, the local notebook generated by this framework is directly used for the MapReduce analysis. Therefore, the full code is not displayed here, but can be seen in source files. The notebook was built this way : after the data was loaded and cleaned, it was converted into an RDD, and a **map** transformation with a lambda function produced key-value pairs: the country as the key, and a tuple of partial statistics (count, sums, squares and product) as the value. Since each function must remain autonomous and isolated, no global variables were used. The tuples were then aggregated with **reduceByKey**, and **mapValues** computed the correlation coefficient from these sums. Thanks to lazy evaluation, the whole pipeline was only executed when **collect** was called.

An output is obtained after the execution of the code, here are its 5 first and last lines.

[9]:
```
# Final results (head and tail)
final_results = sorted(results.collect(), key=lambda x: x[0])

for country, (n, pearson_r) in final_results[:5]:
    print("{}\t{}\t{}".format(
        country, n,
        pearson_r if pearson_r is not None else "NA"
    ))
print()
for country, (n, pearson_r) in final_results[-5:]:
    print("{}\t{}\t{}".format(
        country, n,
        pearson_r if pearson_r is not None else "NA"
    ))
```

```
Afghanistan     58      0.5983417278404136
Albania 49      0.9041163053459342
Algeria 58      0.7212764921189101
Andorra 48      0.3066379401933389
Angola  47      0.8003394898282855

Wallis_and_Futuna       17      NA
Western_Sahara  42      -0.0029736692075012327
Yemen   44      0.9268609766875067
Zambia  47      0.9131633061030852
Zimbabwe        47      0.8097480660046228
```

d) Benchmarking and verdict

The outputs from the three frameworks are the same. However the execution times are different. Indeed, the execution times should be interpreted in the context of each framework's operating environment. Hadoop's execution time is significantly affected by the overhead of starting the MapReduce job and accessing the input file stored in HDFS. In contrast, MongoDB benefits from the fact that the dataset is already imported into the database, avoiding the need for file access during the query execution. PySpark also requires access to the dataset from the notebook's working directory. Therefore, the measured times reflect not only the processing itself but also, to some extent, the overhead associated with data access and framework execution. (***Figure 1***)

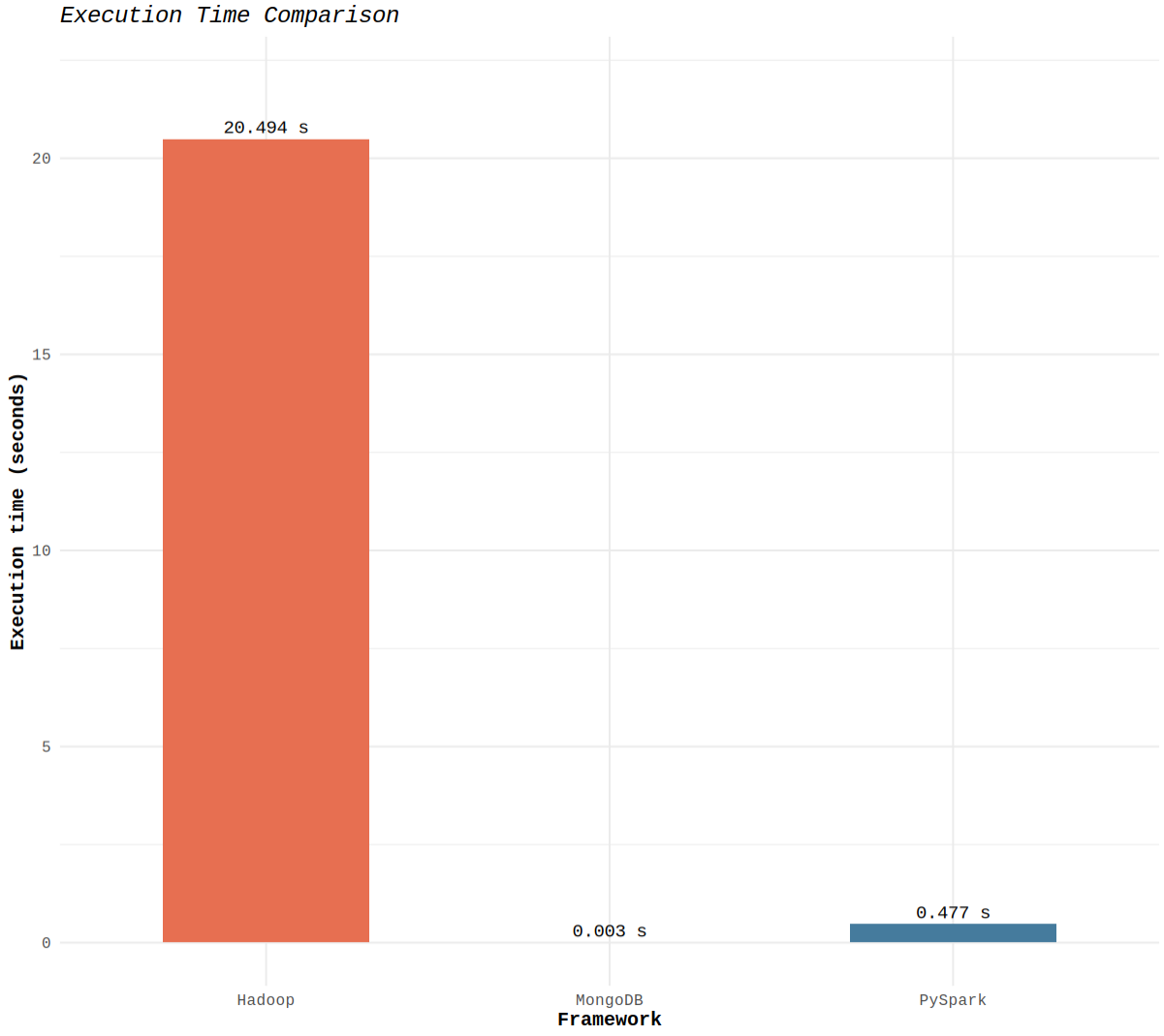


Figure 1. Execution time comparison of Hadoop, MongoDB, and PySpark.
The measured execution times include framework-specific overheads, such as Hadoop job initialization and input file access, while MongoDB operates on data already imported into the database. Also, the times are estimated after 4 runs per framework.

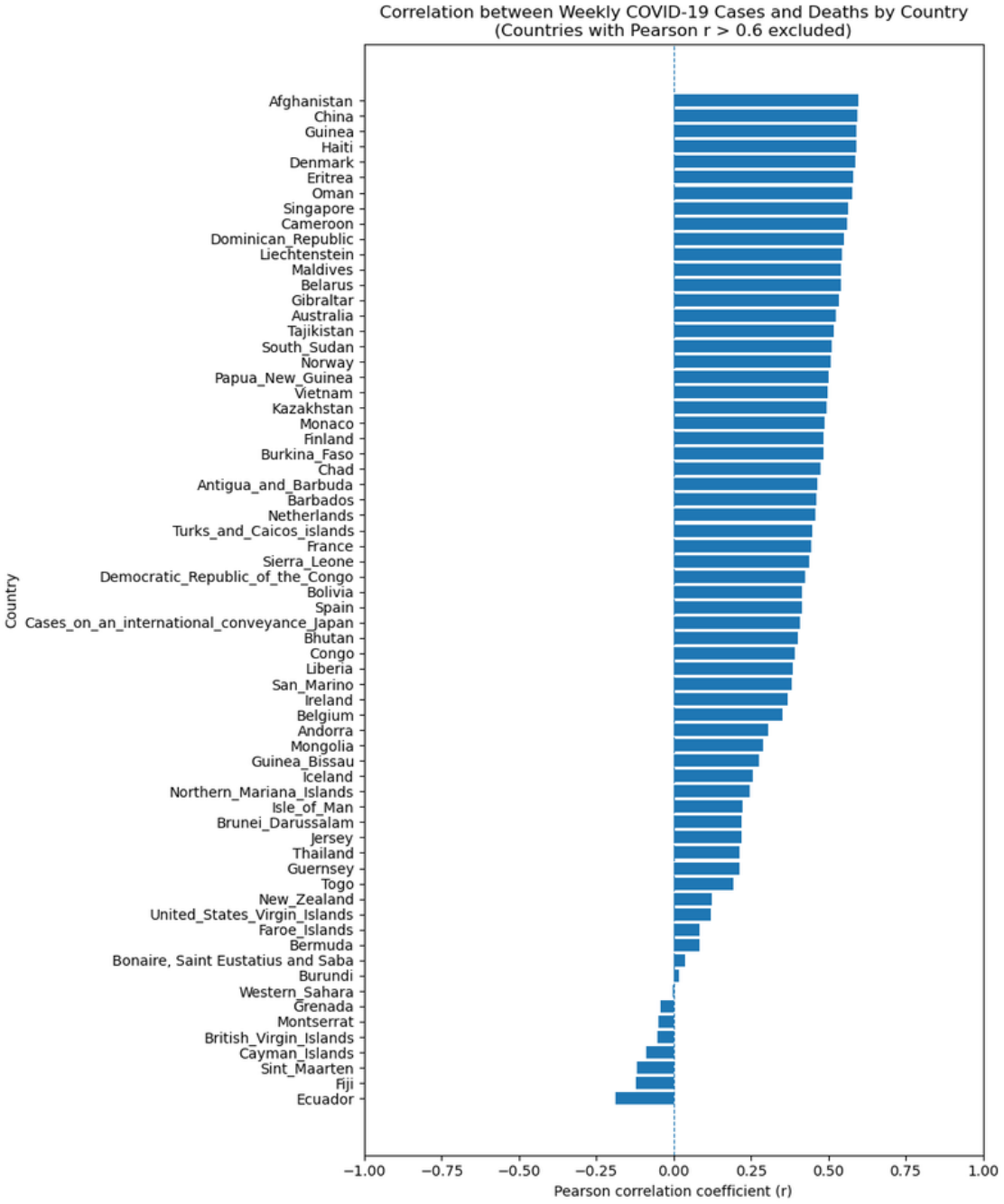


Figure 2. Pearson correlation coefficients between weekly cases and deaths across countries. *149 countries with a correlation coefficient >= 0.6 were excluded to improve figure readability.*

The analysis revealed that out of 215 countries, 149 countries had a Pearson correlation coefficient greater than 0.6, indicating a strong positive association between weekly cases and deaths. In contrast, 8 countries showed a coefficient less than or equal to 0, suggesting no positive linear association between these variables. Ecuador exhibited the lowest correlation, with a Pearson coefficient of -0.25. The relatively weak or negative correlations observed in some countries may be explained by differences in testing capacity, reporting practices, healthcare conditions, and epidemic dynamics. (***Figure 2***)

2.2. Association between population size and COVID-19 cases and deaths

Research question: *To what extent are population size and COVID-19 case and death counts associated across countries?*

The objective is to investigate the association between population size and the total numbers of reported COVID-19 cases and deaths across countries. The same MapReduce logic is implemented in **Hadoop**, **MongoDB** (through Studio 3T), and **PySpark** to compare their outputs and performance.

The Map phase processes each observation and emits the country identifier, its population size, and its weekly case and death counts. During the Shuffle and Sort phase, observations are grouped by country. The Reduce phase aggregates the weekly case and death counts to obtain cumulative totals for each country, while retaining a single population value. Finally, Pearson's correlation coefficient is calculated across countries to assess the association between population size and cumulative COVID-19 cases, and between population size and cumulative deaths, only after the MR job. This produces two correlation coefficients, each describing the relationship between population size and one measure of COVID-19 burden.

a) Hadoop approach

Here are the two corresponding Python scripts used for the MapReduce approach in Hadoop.

```
#!/usr/bin/env python

import sys
import csv
reader = csv.reader(sys.stdin)

for row in reader:
    try:
        if not row or row[0] == "dateRep":
            continue
        country = row[4].strip()
        cases = float(row[2])
        deaths = float(row[3])

        # Population may be missing in some weekly records
        population = row[7].strip()
        if not country:
            continue

        if population == "":
            population = "NA"
        else:
```

```
            population = float(population)
            if population <= 0:
                population = "NA"

        print("{}\t{}\t{}\t{}".format(
            country, population, cases, deaths))

    except (ValueError, IndexError):
        continue
```

Mapper Python file used for this specific Hadoop approach.

```
#!/usr/bin/env python
import sys

current_country = None
total_cases = 0.0
total_deaths = 0.0
population = None
population_mismatch = False


def emit_result(country, cases, deaths, pop, mismatch):
    if country is None:
        return
    if mismatch or pop is None:
        pop = "NA"
    print("{}\t{}\t{}\t{}".format(
        country, cases, deaths, pop
    ))


for line in sys.stdin:
    try:
        country, pop, cases, deaths = line.strip().split("\t")
        cases = float(cases)
        deaths = float(deaths)

        if pop == "NA":
            pop = None
        else:
            pop = float(pop)

    except ValueError:
        continue

    if current_country != country:
        emit_result(
```

```
            current_country,
            total_cases,
            total_deaths,
            population,
            population_mismatch
        )

        current_country = country
        total_cases = 0.0
        total_deaths = 0.0
        population = None
        population_mismatch = False

    total_cases += cases
    total_deaths += deaths

    if pop is not None:
        if population is None:
            population = pop
        elif population != pop:
            population_mismatch = True

emit_result(
    current_country,
    total_cases,
    total_deaths,
    population,
    population_mismatch)
```

Reducer Python file used for this specific Hadoop approach.

An output is obtained using these files with Hadoop approach, here are its 5 first and last lines.

```
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat COVID/MR2/part-* | head -n 5
Afghanistan  55359.0      2413.0    38041757.0
Albania      85336.0      1460.0    2862427.0
Algeria      109088.0     2914.0    43053054.0
Andorra      10251.0      106.0     76177.0
Angola 20086.0      475.0  3    1825299.0

root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat COVID/MR2/part-* | tail -n 5
Wallis_and_Futuna   9.0    0.0    NA
Western_Sahara      766.0  1.0    582458.0
Yemen  2131.0    616.0     29161922.0
Zambia 62633.0      853.0  17861034.0
Zimbabwe     34552.0      1326.0    14645473.0
```

b) MongoDB approach

The following script is executed in Studio3T, with the MapReduce approach.

```
var mapper2 = function() {
    var country = this.countriesAndTerritories;
    var cases = this.cases_weekly;
    var deaths = this.deaths_weekly;
    var population = this.popData2019;

    if (typeof country !== "string" ||
        typeof cases !== "number" ||
        typeof deaths !== "number") {
        return;
    }

    var validPopulation =
        typeof population === "number" && population > 0
        ? population
        : null;

    emit(country, {
        total_cases: cases,
        total_deaths: deaths,
        population: validPopulation
    });
};

var reducer2 = function(key, values) {
    var result = {
        total_cases: 0,
        total_deaths: 0,
        population: null,
        population_mismatch: false
    };

    values.forEach(function(value) {
        result.total_cases += value.total_cases;
        result.total_deaths += value.total_deaths;

        if (value.population !== null) {

            if (result.population === null) {
                result.population = value.population;
            }
            else if (result.population !== value.population) {
                result.population_mismatch = true;
            }
        }
```

```
    });

    if (result.population_mismatch) {
        result.population = null;
    }

    return result;
};

db.covid_weekly.mapReduce(
    mapper2,
    reducer2,
    {
        out: { replace: "MR2_MongoDB" }
    }
);
```

Studio3T file used for this specific MongoDB approach.

An output is obtained using these files with Mongo approach, here are its 5 first and last lines.

{Document id}	total_cases	total_deaths	population
Afghanistan	55359.0	2413.0	38041757.0
Albania	85336.0	1460.0	2862427.0
Algeria	109088.0	2914.0	43053054.0
Andorra	10251.0	106.0	76177.0
Angola	20086.0	475.0	31825299.0

5 first lines of the MongoDB output.

{Document id}	total_cases	total_deaths	population
Wallis_and_Futuna	9.0	0.0	null
Western_Sahara	766.0	1.0	582458.0
Yemen	2131.0	616.0	29161922.0
Zambia	62633.0	853.0	17861034.0
Zimbabwe	34552.0	1326.0	14645473.0

5 last lines of the MongoDB output.

c) PySpark approach

For this approach, the local notebook generated by this framework is directly used for the MapReduce analysis. Therefore, the full code is not displayed here, but can be found in the source files. The notebook follows the same general structure as the previous analysis: after the data was loaded and cleaned, it was converted into an RDD. A **map** transformation with a lambda function produced key-value pairs, using the country as the key and a tuple containing the weekly case count, death count, and population as the value. The tuples were then aggregated using **reduceByKey**, which summed the weekly cases and deaths for each country while checking the consistency of population values across observations. The resulting RDD was converted into a DataFrame containing one aggregated record per country. Thanks to Spark's lazy evaluation, the transformations were only executed when an action such as **collect** was called.

An output is obtained after the execution of the code, here are its 5 first and last lines.

```
for country, total_cases, total_deaths, population in final_df.orderBy("country").limit(5).collect():
    print("{}\t{}\t{}\t{}".format(
        country,
        total_cases,
        total_deaths,
        population if population is not None else "NA"
    ))

print()

for country, total_cases, total_deaths, population in (
    final_df.orderBy("country", ascending=False).limit(5).collect()[::-1]
):
    print("{}\t{}\t{}\t{}".format(
        country,
        total_cases,
        total_deaths,
        population if population is not None else "NA"
    ))
```

```
Afghanistan     55359.0 2413.0  38041757.0
Albania 85336.0 1460.0  2862427.0
Algeria 109088.0        2914.0  43053054.0
Andorra 10251.0 106.0   76177.0
Angola  20086.0 475.0   31825299.0

Wallis_and_Futuna       9.0     0.0     NA
Western_Sahara  766.0   1.0     582458.0
Yemen   2131.0  616.0   29161922.0
Zambia  62633.0 853.0   17861034.0
Zimbabwe        34552.0 1326.0  14645473.0
```

d) Benchmarking and verdict

The outputs produced by the three frameworks are identical, while their execution times differ. This follows the same pattern as in the previous section, where all three frameworks generated equivalent results but exhibited different execution times. (***Figure 3***)

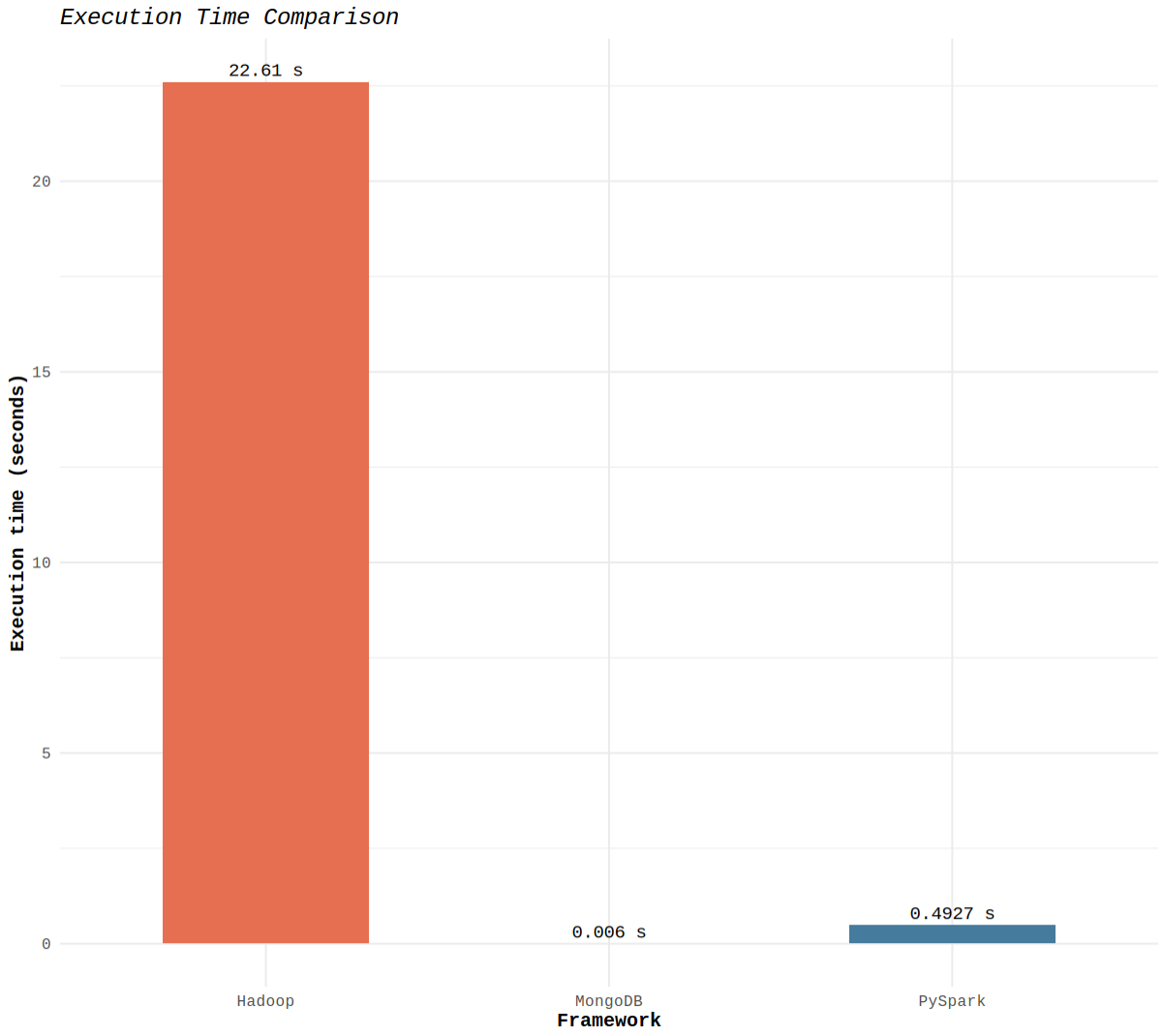


Figure 3. Execution time comparison of Hadoop, MongoDB, and PySpark.
The measured execution times include framework-specific overheads, such as Hadoop job initialization and input file access, while MongoDB operates on data already imported into the database. Also, the times are estimated after 4 runs per framework.

Regarding the correlation, Pearson's analysis was done after the prior MapReduce job. It revealed a moderate positive association between population size and total reported COVID-19 cases (r = 0.3977). A similar, slightly weaker association was observed between population size and total reported deaths (r = 0.3567). These results suggest that more populated countries tend to report higher absolute numbers of cases and deaths. However, the moderate strength of both correlations indicates that population size alone does not account for the differences observed across countries. Other factors, such as epidemic dynamics, public health measures, testing capacity, and demographic characteristics, may also contribute to these variations. It is important to note that these correlations concern absolute counts rather than population-normalized indicators. They therefore do not establish that individuals living in more populated countries faced a higher risk of infection or death. (***Figure 4***)

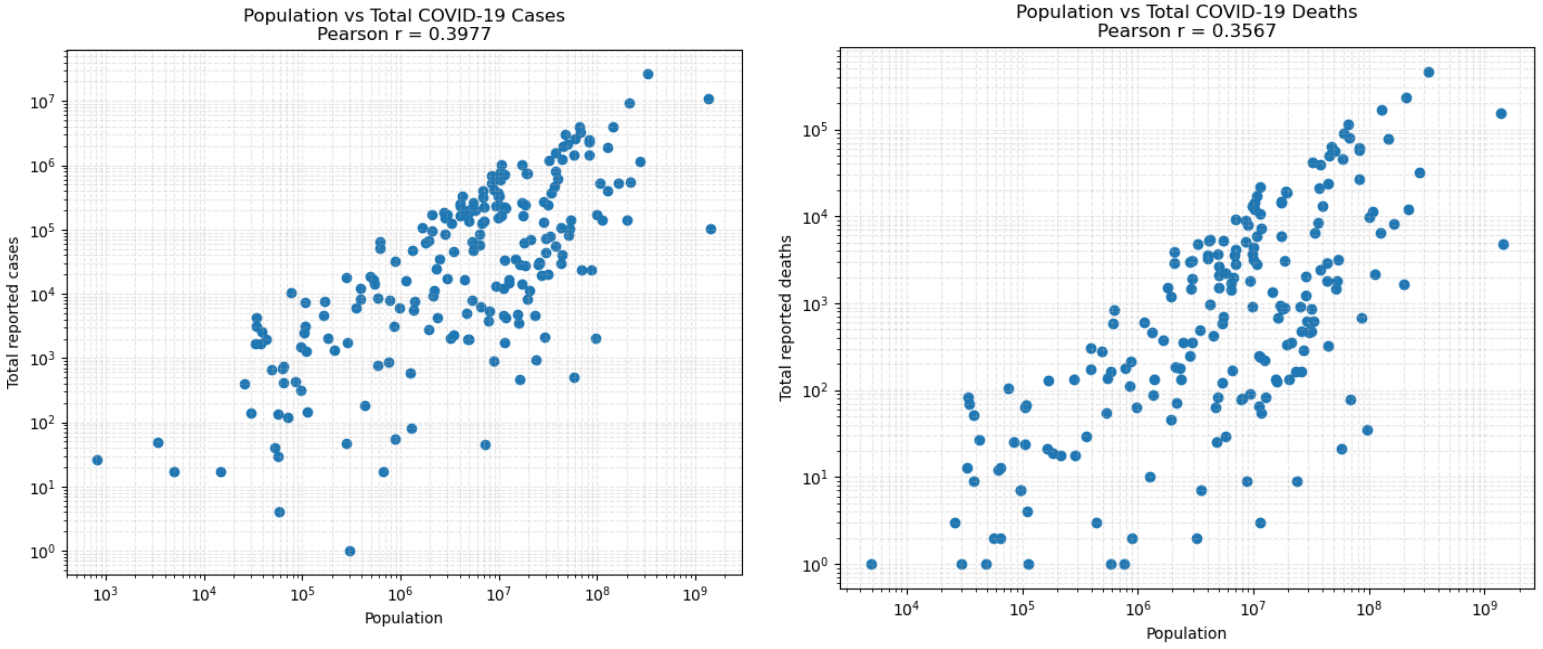


Figure 4. Relationship between country population size and cumulative cases and deaths. *COVID-19 cases (left) and deaths (right). Both axes are displayed on a logarithmic scale. Pearson correlation coefficients were calculated using the original, non-transformed values, obtained from the MapReduce approach.*

2.3. Differences in cumulative deaths-to-cases ratios across countries

Research question: *To what extent does the cumulative deaths-to-cases ratio vary across countries?*

The objective is to investigate differences in the proportion of reported COVID-19 deaths relative to reported cases across countries. The same MapReduce logic is implemented in **Hadoop**, **MongoDB** (through Studio 3T), and **PySpark** to compare their outputs and performance.

The Map phase processes each observation and emits the country identifier along with its weekly case and death counts. During the Shuffle and Sort phase, observations are grouped by country. The Reduce phase aggregates the weekly case and death counts to obtain cumulative totals for each country. Finally, the cumulative deaths-to-cases ratio is calculated for each country by dividing its total reported deaths by its total reported cases and multiplying the result by 100. This produces one ratio per country, allowing comparisons of the relative proportion of reported deaths across countries.

a) Hadoop approach

Here are the two corresponding Python scripts used for the MapReduce approach in Hadoop.

```python
#!/usr/bin/env python
import sys
import csv

reader = csv.reader(sys.stdin)
for row in reader:
    try:
        if not row or row[0] == "dateRep":
            continue

        country = row[4].strip()
        cases = float(row[2])
        deaths = float(row[3])
        if not country:
            continue

        print("{}\t{}\t{}".format(country, cases, deaths))

    except (ValueError, IndexError):
        continue
```

Mapper Python file used for this specific Hadoop approach.

```python
#!/usr/bin/env python
import sys

current_country = None
total_cases = 0.0
total_deaths = 0.0


def emit_result(country, cases, deaths):
    if country is None:
        return

    if cases == 0:
        ratio = "NA"
    else:
        ratio = (deaths / cases) * 100

    print("{}\t{}\t{}\t{}".format(country, cases, deaths, ratio))

for line in sys.stdin:
    try:
        country, cases, deaths = line.strip().split("\t")
        cases = float(cases)
        deaths = float(deaths)

    except ValueError:
        continue

    if current_country != country:
        emit_result(
            current_country,
            total_cases,
            total_deaths
        )

        current_country = country
        total_cases = 0.0
        total_deaths = 0.0

    total_cases += cases
    total_deaths += deaths


emit_result(
    current_country,
    total_cases,
    total_deaths)
```

Reducer Python file used for this specific Hadoop approach.

An output is obtained using these files with Hadoop approach, here are its 5 first and last lines.

```
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat COVID/MR3/part-* |
head -n 5
Afghanistan    55359.0    2413.0    4.3588215105
Albania    85336.0    1460.0    1.71088403487
Algeria    109088.0    2914.0    2.67123789968
Andorra    10251.0    106.0    1.03404545898
Angola    20086.0    475.0    2.36483122573

root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat COVID/MR3/part-* |
tail -n 5
Wallis_and_Futuna    9.0    0.0    0.0
Western_Sahara    766.0    1.0    0.130548302872
Yemen    2131.0    616.0    28.9066166119
Zambia    62633.0    853.0    1.36190187281
Zimbabwe    34552.0    1326.0    3.83769391063
```

b) MongoDB approach

The following script is executed in Studio3T, with the MapReduce approach.

```
var mapper3 = function() {
    var country = this.countriesAndTerritories;
    var cases = this.cases_weekly;
    var deaths = this.deaths_weekly;
    if (typeof country !== "string" ||
        typeof cases !== "number" ||
        typeof deaths !== "number") {
        return;
    }
    emit(country, {
        total_cases: cases,
        total_deaths: deaths
    });
};

var reducer3 = function(key, values) {
    var result = {
        total_cases: 0,
        total_deaths: 0
    };
    values.forEach(function(value) {
        result.total_cases += value.total_cases;
```

```
        result.total_deaths += value.total_deaths;
    });
    return result;
};

db.covid_weekly.mapReduce(
    mapper3,
    reducer3,
    {
        out: {
            replace: "MR3_MongoDB"
        },
        finalize: function(key, value) {
            value.deaths_to_cases_ratio =
                value.total_cases === 0
                ? null
                : (value.total_deaths / value.total_cases) * 100;
            return value;
        }
    }
);
```

Studio3T file used for this specific MongoDB approach.

An output is obtained using these files with Mongo approach, here are its 5 first and last lines.

{Document id}	total_cases	total_deaths	deaths_to_cases_ratio
Afghanistan	55359.0	2413.0	4.358821510504163
Albania	85336.0	1460.0	1.71088403487391
Algeria	109088.0	2914.0	2.6712378996773247
Andorra	10251.0	106.0	1.0340454589796118
Angola	20086.0	475.0	2.3648312257293638

5 first lines of the MongoDB output.

{Document id}	total_cases	total_deaths	deaths_to_cases_ratio
Wallis_and_Futuna	9.0	0.0	0.0
Western_Sahara	766.0	1.0	0.13054830287206268
Yemen	2131.0	616.0	28.90661661191929
Zambia	62633.0	853.0	1.3619018728146506
Zimbabwe	34552.0	1326.0	3.83769391062746

5 last lines of the MongoDB output.

c) PySpark approach

For this approach, the local notebook generated by this framework is directly used for the MapReduce analysis. Therefore, the full code is not displayed here, but can be found in the source files. The notebook follows the same general structure as the previous analysis: after the data was loaded and cleaned, it was converted into an RDD. A **map** transformation with a lambda function produced key-value pairs, using the country as the key and a tuple containing the weekly case and death counts as the value. The tuples were then aggregated using **reduceByKey**, which summed the weekly cases and deaths for each country. The resulting RDD was used to calculate the cumulative deaths-to-cases ratio for each country and then converted into a DataFrame containing one aggregated record per country. Thanks to Spark's lazy evaluation, the transformations were only executed when an action such as **collect** was called.

An output is obtained after the execution of the code, here are its 5 first and last lines.

```
# First five countries
first_five = (final_df.orderBy("country").limit(5).collect())
for row in first_five:
    print(
        "{}\t{}\t{}\t{}".format(row["country"],row["total_cases"],row["total_deaths"],row["deaths_to_cases_ratio"])
    )
print()

# Last five countries
last_five = (final_df.orderBy("country", ascending=False).limit(5).collect()[::-1])
for row in last_five:
    print(
        "{}\t{}\t{}\t{}".format(row["country"],row["total_cases"],row["total_deaths"],row["deaths_to_cases_ratio"])
    )
```

```
Afghanistan     55359.0 2413.0  4.358821510504163
Albania 85336.0 1460.0  1.71088403487391
Algeria 109088.0        2914.0  2.6712378996773247
Andorra 10251.0 106.0   1.0340454589796118
Angola  20086.0 475.0   2.3648312257293638

Wallis_and_Futuna       9.0     0.0     0.0
Western_Sahara  766.0   1.0     0.13054830287206268
Yemen   2131.0  616.0   28.90661661191929
Zambia  62633.0 853.0   1.3619018728146506
Zimbabwe        34552.0 1326.0  3.83769391062746
```

d) Benchmarking and verdict

The outputs produced by the three frameworks are identical, while their execution times differ. This follows the same pattern as in the previous section, where all three frameworks generated equivalent results but exhibited different execution times. (***Figure 5***)

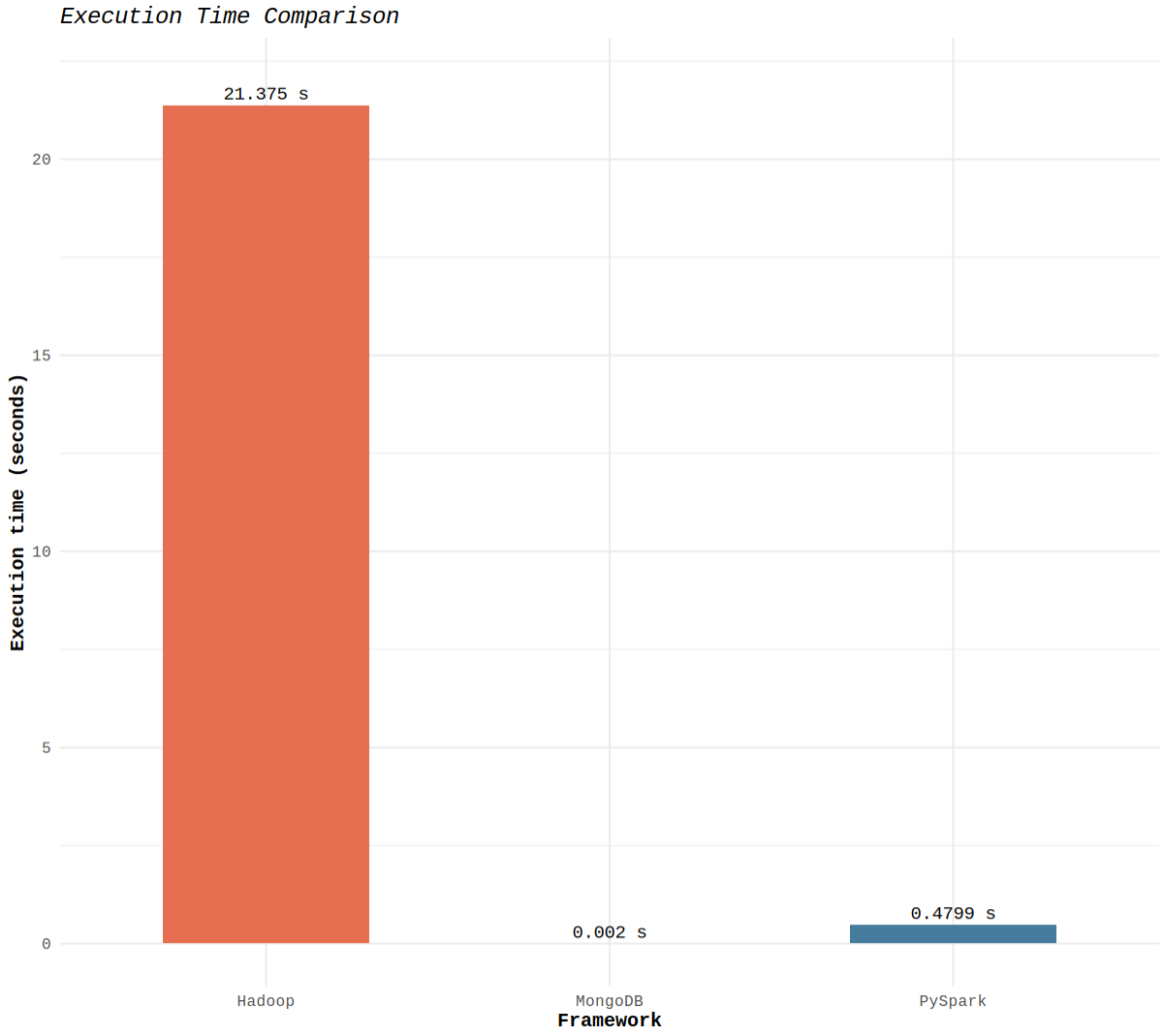


Figure 5. Execution time comparison of Hadoop, MongoDB, and PySpark.
The measured execution times include framework-specific overheads, such as Hadoop job initialization and input file access, while MongoDB operates on data already imported into the database. Also, the times are estimated after 4 runs per framework.

The analysis revealed substantial variation in cumulative deaths-to-cases ratios across countries. Among the 20 countries selected for the lowest and highest ratios, ten reported no deaths despite having recorded cases, resulting in a ratio of 0%. These included Laos (45 cases), Cambodia (474 cases), and New Caledonia (47 cases). At the other end of the distribution, Yemen exhibited the highest ratio (28.91%), with 616 reported deaths out of 2,131 cases. These results highlight considerable differences in the proportion of reported deaths relative to reported cases across countries. However, the ratios should be interpreted cautiously, as they may be influenced by differences in testing capacity, reporting practices, healthcare systems, demographic characteristics, and epidemic timing. Furthermore, countries with relatively few reported cases may exhibit particularly high or low ratios due to their small case counts. Therefore, these values should not be interpreted as direct measurements of infection fatality risk or as definitive indicators of differences in healthcare performance. (***Figure 6***)

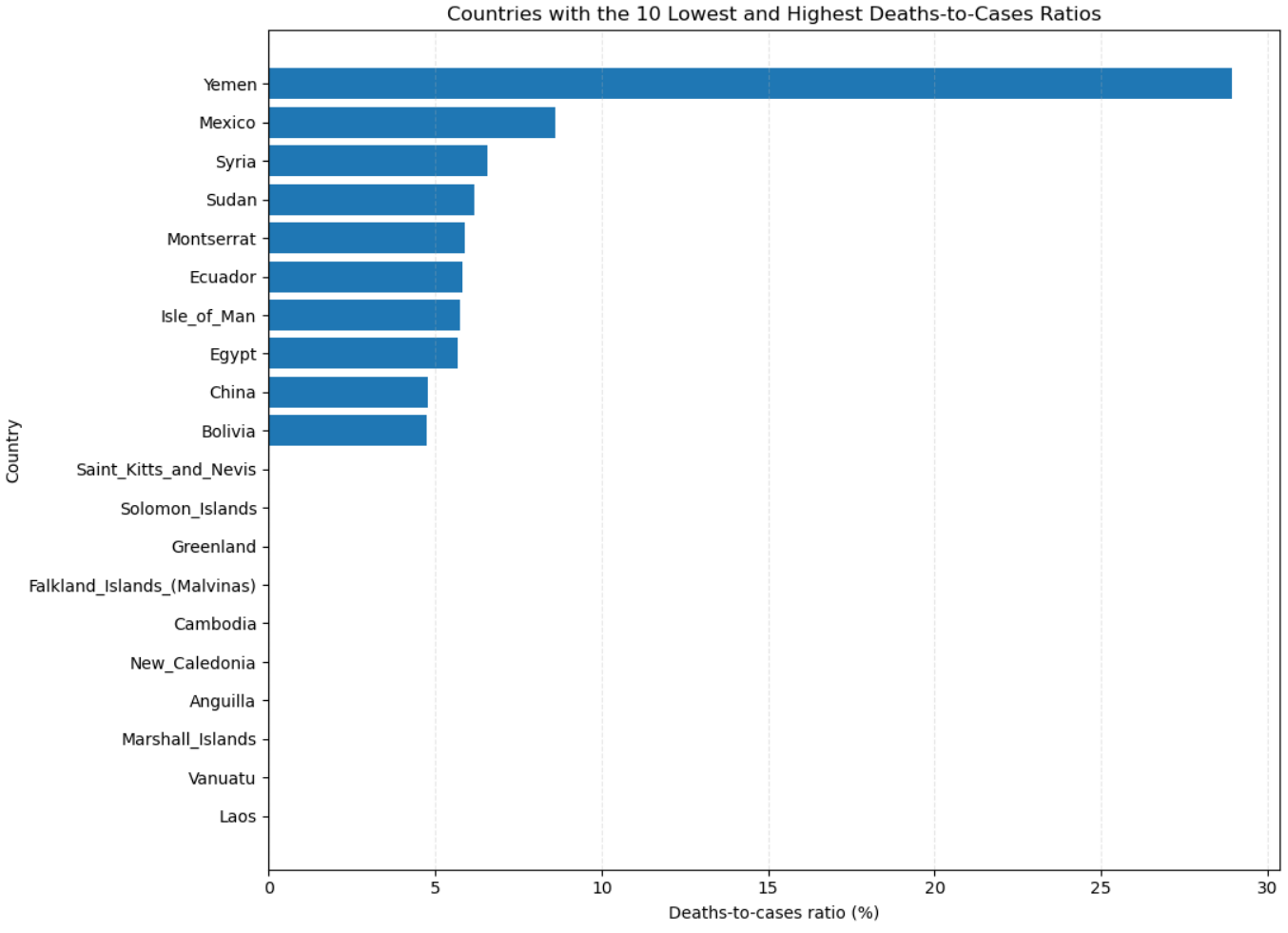


Figure 6. Countries with the 10 lowest and 10 highest cumulative deaths-to-cases ratios.
The horizontal barplot compares the proportion of reported deaths relative to reported cases across the selected countries.

2.4. Continental differences in reported COVID-19 cases and deaths

Research question: *How did the total numbers of reported COVID-19 cases and deaths differ across continents?*

The objective is to investigate differences in the total numbers of reported COVID-19 cases and deaths across continents. The same MapReduce logic is implemented in **Hadoop**, **MongoDB** (through Studio 3T), and **PySpark** to compare their outputs and performance.

The Map phase processes each observation and emits the continent identifier along with its weekly case and death counts. During the Shuffle and Sort phase, observations are grouped by continent. The Reduce phase aggregates the weekly case and death counts to obtain cumulative totals for each continent. This produces one aggregated record per continent, allowing comparisons of the total reported numbers of cases and deaths across continents.

a) Hadoop approach

Here are the two corresponding Python scripts used for the MapReduce approach in Hadoop.

```
#!/usr/bin/env python

import sys
import csv

reader = csv.reader(sys.stdin)
for row in reader:
    try:
        if not row or row[0] == "dateRep":
            continue

        continent = row[9].strip()
        cases = float(row[2])
        deaths = float(row[3])

        if not continent:
            continue

        print("{}\t{}\t{}".format(
            continent, cases, deaths
        ))

    except (ValueError, IndexError):
        continue
```

Mapper Python file used for this specific Hadoop approach.

```python
#!/usr/bin/env python
import sys

current_continent = None
total_cases = 0.0
total_deaths = 0.0

def emit_result(continent, cases, deaths):
    if continent is None:
        return

    print("{}\t{}\t{}".format(
        continent, cases, deaths
    ))

for line in sys.stdin:
    try:
        continent, cases, deaths = line.strip().split("\t")
        cases = float(cases)
        deaths = float(deaths)
    except ValueError:
        continue

    if current_continent != continent:
        emit_result(current_continent, total_cases, total_deaths)

        current_continent = continent
        total_cases = 0.0
        total_deaths = 0.0

    total_cases += cases
    total_deaths += deaths

emit_result(current_continent, total_cases, total_deaths)
```

Reducer Python file used for this specific Hadoop approach.

An output is obtained using these files with the Hadoop approach, as shown below.

```
Africa    3673181.0    95128.0
America    47620931.0    1107060.0
Asia    20438608.0    343886.0
Europe    34642469.0    775857.0
Oceania    57809.0    1210.0
Other    705.0    6.0
```

b) MongoDB approach

The following script is executed in Studio3T, with the MapReduce approach.

```
var mapper4 = function() {
    var continent = this.continentExp;
    var cases = this.cases_weekly;
    var deaths = this.deaths_weekly;

    if (typeof continent !== "string" ||
        typeof cases !== "number" ||
        typeof deaths !== "number") {
        return;
    }

    emit(continent, {
        total_cases: cases,
        total_deaths: deaths
    });
};

var reducer4 = function(key, values) {
    var result = {
        total_cases: 0,
        total_deaths: 0
    };

    values.forEach(function(value) {
        result.total_cases += value.total_cases;
        result.total_deaths += value.total_deaths;
    });

    return result;
};

db.covid_weekly.mapReduce(
    mapper4,
    reducer4,
    {
        out: { replace: "MR4_MongoDB" }
    }
);
```

Studio3T file used for this specific MongoDB approach.

An output is obtained using these files with the Mongo approach, as shown below.

{Document id}	total_cases	total_deaths
Africa	3673181.0	95128.0
America	47620931.0	1107060.0
Asia	20438608.0	343886.0
Europe	34642469.0	775857.0
Oceania	57809.0	1210.0
Other	705.0	6.0

Entire MongoDB output.

c) PySpark approach

For this approach, the local notebook generated by this framework is directly used for the MapReduce analysis. Therefore, the full code is not displayed here, but can be found in the source files. The notebook follows the same general structure as the previous analysis: after the data was loaded and cleaned, it was converted into an RDD. A **map** transformation produced key-value pairs, using the continent as the key and a tuple containing the weekly case and death counts as the value. The tuples were then aggregated using **reduceByKey**, which summed the weekly cases and deaths for each continent. The resulting RDD was converted into a DataFrame containing one aggregated record per continent. Thanks to Spark's lazy evaluation, the transformations were only executed when an action such as **collect** was called.

Here is the output obtained after the execution of the code.

```
final_df = final_df.orderBy("continent").collect()

for row in final_df:
    print(
        "{}\t{}\t{}".format(
            row["continent"],
            row["total_cases"],
            row["total_deaths"]
        )
    )
```

```
Africa  3673181.0       95128.0
America 47620931.0      1107060.0
Asia    20438608.0      343886.0
Europe  34642469.0      775857.0
Oceania 57809.0 1210.0
Other   705.0   6.0
```

d) Benchmarking and verdict

From this section onwards, the benchmark focuses on the memory usage and the CPU usage, as the execution times follow the same patterns. The outputs produced by the three frameworks are identical. Before execution, all three are nearly idle on CPU, at 1.3-3.5% of a single core. During execution, CPU rises sharply for Hadoop (558%, about 5.6 cores) and PySpark (279%, about 2.8 cores), while MongoDB stays modest at 23%. On a 16-core host, even Hadoop uses only about a third of the available CPU. Memory shows a different pattern: it barely changes for PySpark (14.5% to 15.1%) or MongoDB (about 0.5%), because PySpark already holds about 2.1 GiB while idle. Hadoop is the only container with a clear memory increase, from 10.5% to 14.5% of the 14.96 GiB limit. Overall, the benchmarks are mainly CPU-bound, with Hadoop the most demanding of the three. (***Figure 7***)

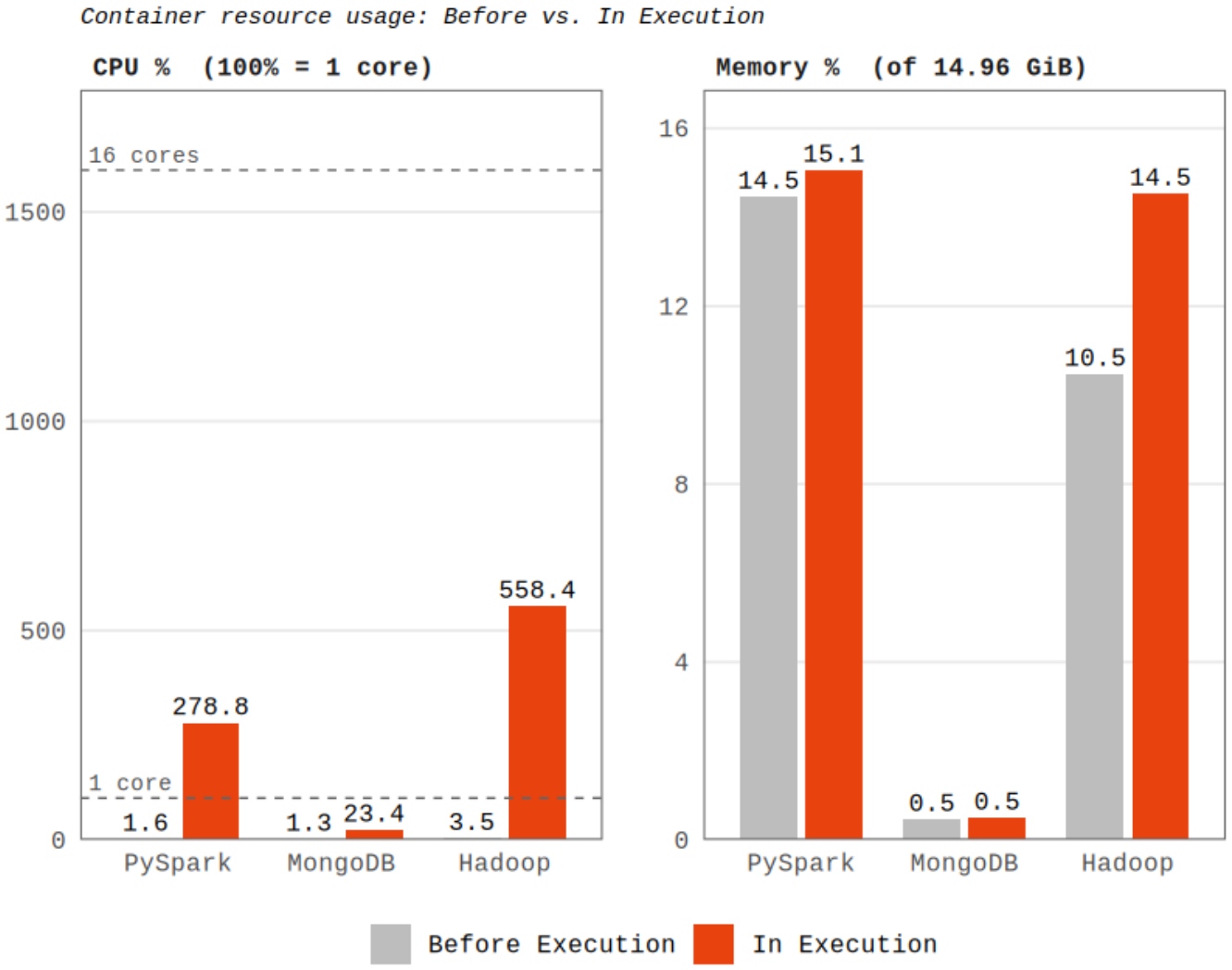


Figure 7. CPU and memory usage of the frameworks before and during the execution.
The measured execution CPU and memory usages include framework-specific overheads, such as Hadoop job initialization and input file access, while MongoDB operates on data already imported into the database. Also, the usages on average are estimated after 4 runs per framework.

The analysis revealed substantial differences in cumulative reported COVID-19 cases and deaths across continents. *America* recorded the highest totals, with 47,620,931 cases and 1,107,060 deaths, followed by *Europe* (34,642,469 cases and 775,857 deaths) and *Asia* (20,438,608 cases and 343,886 deaths). *Africa* reported 3,673,181 cases and 95,128 deaths, while Oceania recorded considerably lower totals, with 57,809 cases and 1,210 deaths. The *Other* category contained only 705 cases and 6 deaths. These results highlight the geographical variation in the absolute numbers of reported COVID-19 cases and deaths over the study period. However, these differences should be interpreted cautiously, as they may partly reflect variations in population size, testing capacity, reporting practices, and epidemic dynamics across continents. Therefore, absolute case and death counts should not be interpreted as direct measures of individual infection or mortality risk. Overall, the number of reported cases was substantially higher than the number of reported deaths across all continents. However, this difference in absolute counts does not establish an association between cases and deaths. Their relationship requires a separate statistical analysis, as explored in the 1st study.

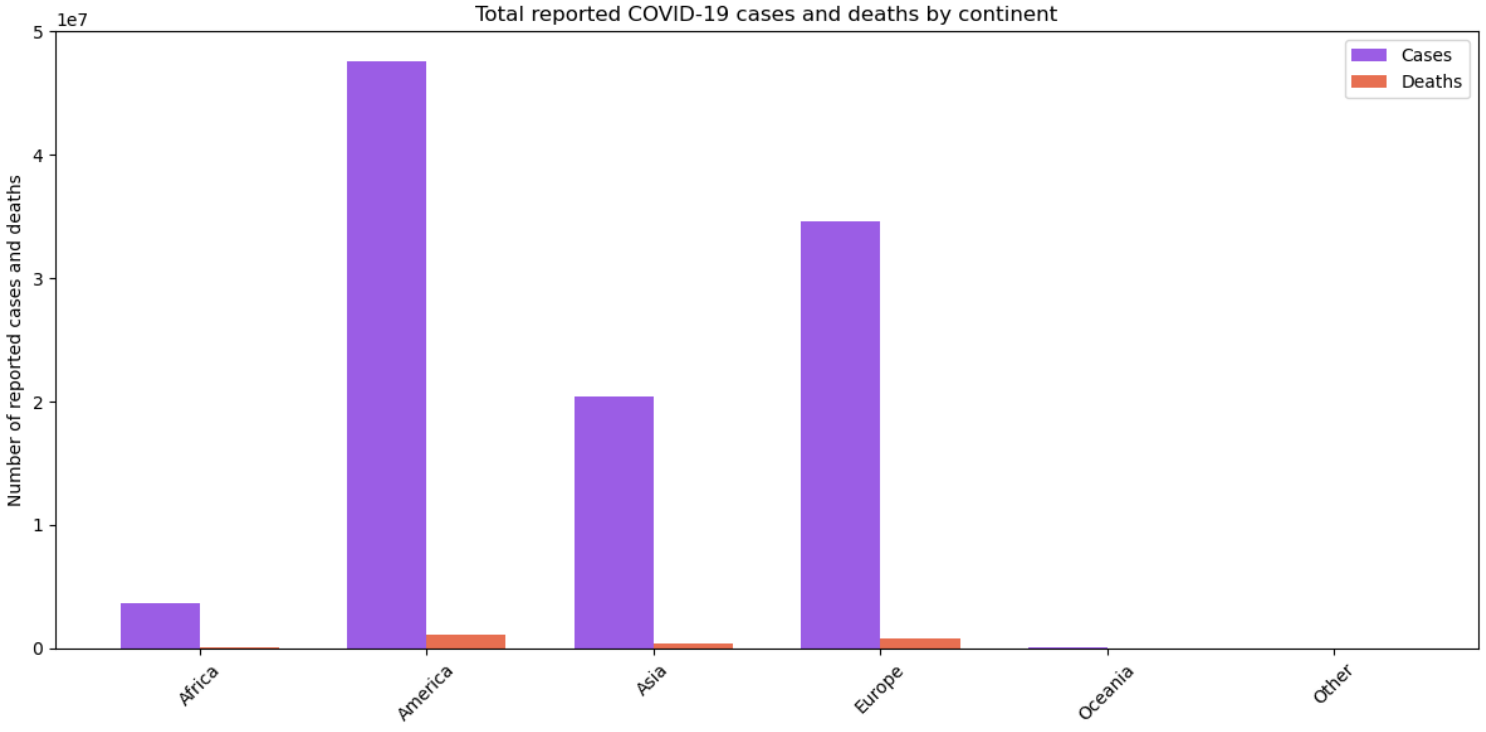


Figure 8. Cumulative reported COVID-19 cases and deaths across continents.
The grouped bar plot compares the total numbers of reported cases and deaths for each continent over the study period.

2.5. Identification of peak COVID-19 weeks across continents

Research question: *During which weeks did each continent record its highest numbers of weekly COVID-19 cases and deaths?*

The objective is to identify the weeks during which each continent recorded its highest numbers of reported COVID-19 cases and deaths. The same MapReduce logic is implemented in **Hadoop**, **MongoDB** (through Studio 3T), and **PySpark** to compare their outputs and performance.

During the Map phase, each observation emits the continent identifier as the key, along with the corresponding week and weekly case and death counts as values. During the Shuffle and Sort phase, all observations are grouped by continent. The Reduce phase processes the weekly observations for each continent, aggregates the case and death counts by week, and identifies the weeks with the highest weekly totals for each indicator. This produces one aggregated record per continent, containing the peak week and corresponding number of cases and deaths.

a) Hadoop approach

Here are the two corresponding Python scripts used for the MapReduce approach in Hadoop.

```python
#!/usr/bin/env python
import sys
import csv

reader = csv.reader(sys.stdin)
for row in reader:
    try:
        if not row or row[0] == "dateRep":
            continue
        week = row[1].strip()
        cases = float(row[2])
        deaths = float(row[3])
        continent = row[8].strip()

        if not continent or not week:
            continue
        print("{}\t{}\t{}\t{}".format(continent, week, cases, deaths))

    except (ValueError, IndexError):
        continue
```

Mapper Python file used for this specific Hadoop approach.

```python
#!/usr/bin/env python
import sys

current_continent = None
weekly_totals = {}

def emit_result(continent, weekly_totals):
    if continent is None or not weekly_totals:
        return
    peak_cases = max(v[0] for v in weekly_totals.values())
    peak_deaths = max(v[1] for v in weekly_totals.values())

    peak_cases_weeks = [
        week for week, values in weekly_totals.items()
        if values[0] == peak_cases
    ]
    peak_deaths_weeks = [
        week for week, values in weekly_totals.items()
        if values[1] == peak_deaths
    ]

    print("{}\t{}\t{}\t{}\t{}".format(
        continent,
        ",".join(sorted(peak_cases_weeks)),
        peak_cases,
        ",".join(sorted(peak_deaths_weeks)),
        peak_deaths))

for line in sys.stdin:
    try:
        continent, week, cases, deaths = line.strip().split("\t")
        cases = float(cases)
        deaths = float(deaths)

    except ValueError:
        continue

    if current_continent != continent:
        emit_result(current_continent, weekly_totals)
        current_continent = continent
        weekly_totals = {}

    if week not in weekly_totals:
        weekly_totals[week] = [0.0, 0.0]
    weekly_totals[week][0] += cases
    weekly_totals[week][1] += deaths

emit_result(current_continent, weekly_totals)
```

Reducer Python file used for this specific Hadoop approach.

An output is obtained using these files with the Hadoop approach, as shown below.

```
root@15031c27bfa3:/usr/local/hadoop# ./bin/hdfs dfs -cat COVID/MR5/part-*
Africa    2021-01    227221.0    2021-03    6501.0
America    2021-01    2699838.0    2021-03    48277.0
Asia    2020-38    851577.0    2020-38    12466.0
Europe    2020-45    2092934.0    2021-03    39948.0
Oceania    2020-31    3607.0    2020-36    148.0
Other    2020-08    336.0    2020-08,2020-09    3.0
```

b) MongoDB approach

The following script is executed in Studio3T, with the MapReduce approach.

```
var mapper5 = function() {
    var continent = this.continentExp;
    var week = this.year_week;
    var cases = this.cases_weekly;
    var deaths = this.deaths_weekly;

    if (typeof continent !== "string" ||
        typeof week !== "string" ||
        typeof cases !== "number" ||
        typeof deaths !== "number") {
        return;
    }

    emit(continent, {
        week: week,
        cases: cases,
        deaths: deaths
    });
};

var reducer5 = function(key, values) {
    var weeklyTotals = {};
    values.forEach(function(value) {
        var week = value.week;

        if (!weeklyTotals[week]) {
            weeklyTotals[week] = {
                cases: 0,
                deaths: 0
            };
        }
        weeklyTotals[week].cases += value.cases;
```

```
        weeklyTotals[week].deaths += value.deaths;
    });

    var result = {
        peak_cases: 0,
        peak_cases_weeks: [],
        peak_deaths: 0,
        peak_deaths_weeks: []
    };

    Object.keys(weeklyTotals).forEach(function(week) {
        var cases = weeklyTotals[week].cases;
        var deaths = weeklyTotals[week].deaths;

        if (cases > result.peak_cases) {
            result.peak_cases = cases;
            result.peak_cases_weeks = [week];
        } else if (cases === result.peak_cases) {
            result.peak_cases_weeks.push(week);
        }

        if (deaths > result.peak_deaths) {
            result.peak_deaths = deaths;
            result.peak_deaths_weeks = [week];
        } else if (deaths === result.peak_deaths) {
            result.peak_deaths_weeks.push(week);
        }
    });
    return result;
};

db.covid_weekly.mapReduce(
    mapper5,
    reducer5,
    {
        out: { replace: "MR5_MongoDB" }
    }
);
```

Studio3T file used for this specific MongoDB approach.

An output is obtained using these files with the Mongo approach, as shown below.

_id	peak_cases_week	peak_cases	peak_deaths_week	peak_deaths
Africa	2021-01	227221.0	2021-03	6501.0
America	2021-01	2699838.0	2021-03	48277.0
Asia	2020-38	851577.0	2020-38	12466.0
Europe	2020-45	2092934.0	2021-03	39948.0
Oceania	2020-31	3607.0	2020-36	148.0
Other	2020-08	336.0	2020-08	3.0

Entire MongoDB output.

c) PySpark approach

For this approach, the local notebook generated by this framework is directly used for the MapReduce analysis. Therefore, the full code is not displayed here, but can be found in the source files. The notebook follows the same general structure as the previous analyses: after the data was loaded and cleaned, it was converted into an RDD. A **map** transformation produced key-value pairs, using the continent as the key and a tuple containing the week, weekly case count, and weekly death count as the value. The tuples were then grouped by continent through **reduceByKey**, which aggregated the weekly case and death counts for each continent. Within each continent, the weekly totals were compared to identify the weeks with the highest numbers of cases and deaths. The resulting data was converted into a DataFrame containing one record per continent, with the peak weeks and their corresponding case and death counts. Thanks to Spark's lazy evaluation, the transformations were only executed when an action such as **collect** was called.

Here is the output obtained after the execution of the code.

```
final_df = final_df.orderBy("continent").collect()

for row in final_df:
    print(
        "{}\t{}\t{}\t{}\t{}".format(
            row["continent"],
            row["peak_cases_weeks"],
            row["peak_cases"],
            row["peak_deaths_weeks"],
            row["peak_deaths"]
        )
    )
```

```
Africa  2021-01 227221.0        2021-03 6501.0
America 2021-01 2699838.0       2021-03 48277.0
Asia    2020-38 851577.0        2020-38 12466.0
Europe  2020-45 2092934.0       2021-03 39948.0
Oceania 2020-31 3607.0  2020-36 148.0
Other   2020-08 336.0   2020-08, 2020-09        3.0
```

d) Benchmarking and verdict

The outputs produced by the three frameworks are identical. Before execution, all three are nearly idle on CPU, at 1.3-3.5% of a single core. During execution, CPU rises sharply for PySpark (1106%, about 11 cores) and Hadoop (297%, about 3 cores), while MongoDB stays modest at 26%. On a 16-core host, PySpark uses about 69% of the available CPU, and Hadoop about 19%. Memory increases for all three, but by different amounts: PySpark rises from 14.5% to 22.6% of the 14.96 GiB limit (about 1.2 GiB more), Hadoop from 10.5% to 14.6% (about 0.6 GiB more), and MongoDB barely moves, from 0.46% to 0.67%. Overall, the benchmarks are mainly CPU-bound, with PySpark the most demanding of the three. (***Figure 9***)

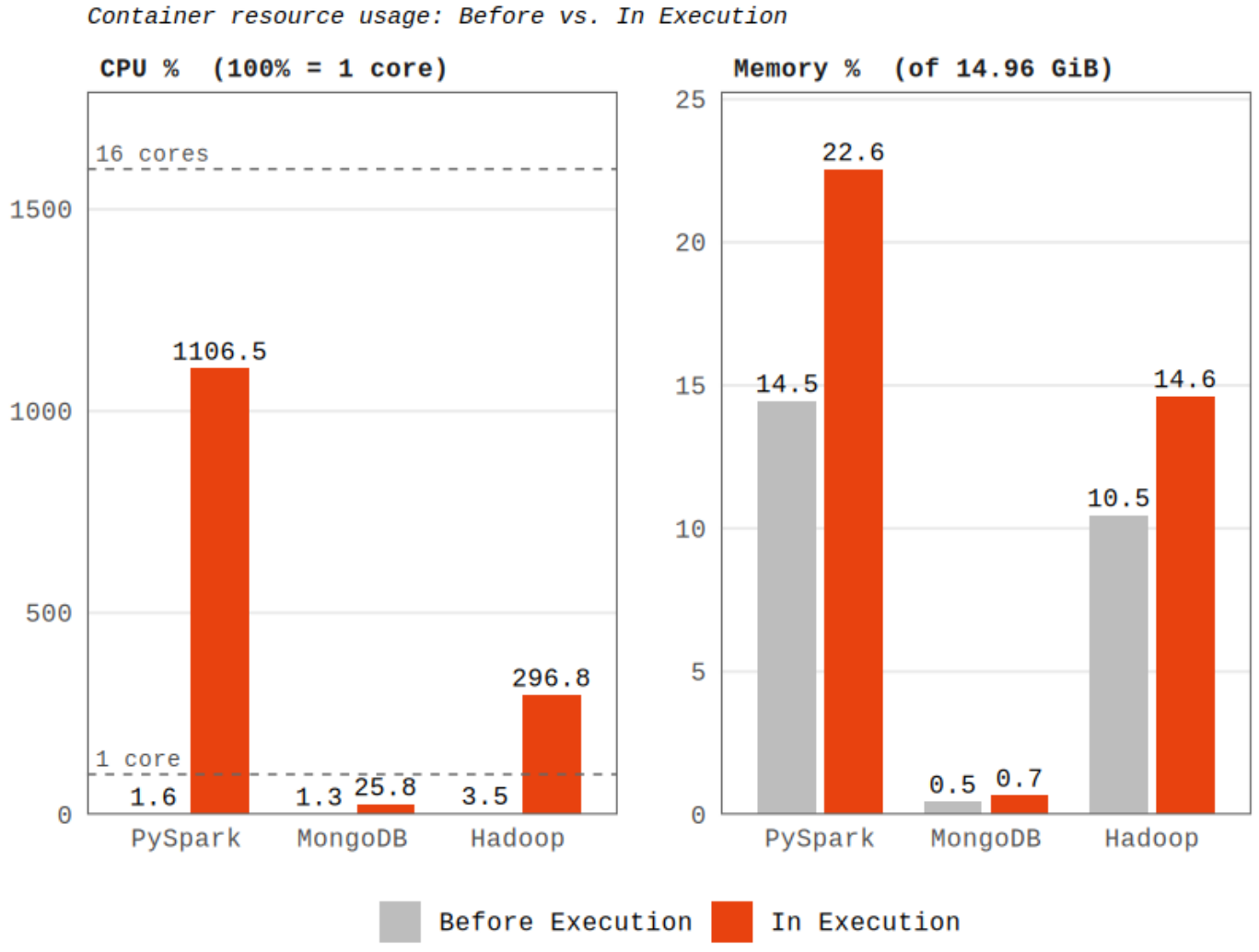


Figure 9. CPU and memory usage of the frameworks before and during the execution.
The measured execution CPU and memory usages include framework-specific overheads, such as Hadoop job initialization and input file access, while MongoDB operates on data already imported into the database. Also, the usages on average are estimated after 4 runs per framework.

The analysis revealed substantial differences in the timing and magnitude of weekly COVID-19 peaks across continents. *America* recorded the highest peak in weekly cases, with 2,699,838 cases during week 2021-01, followed by *Europe* (2,092,934 cases during week 2020-45) and *Asia* (851,577 cases during week 2020-38). *Africa* reached its highest weekly case count during week 2021-01, with 227,221 cases, while *Oceania* recorded a considerably lower peak of 3,607 cases during week 2020-31. The *Other* category reached its peak during week 2020-08, with 336 cases. Regarding deaths, *America* also recorded the highest weekly peak, with 48,277 deaths during week 2021-03, followed by *Europe* (39,948 deaths during the same week) and *Asia* (12,466 deaths during week 2020-38). *Africa* reached its highest weekly death count during week 2021-03, with 6,501 deaths, whereas *Oceania* recorded its peak during week 2020-36, with 148 deaths. The *Other* category recorded only 3 deaths, with the same maximum observed during weeks 2020-08 and 2020-09.

An interesting observation is that the peak in weekly deaths occurred approximately two weeks after the peak in weekly cases in both Africa and America. This temporal difference may reflect the delay between infection, disease progression, and death. However, this pattern was not consistent across all continents. In Asia, both peaks occurred during the same week, while Oceania recorded a five-week delay and Europe an eighteen-week delay. These differences suggest that the temporal relationship between reported cases and deaths varied across continents. (***Figure 10***)

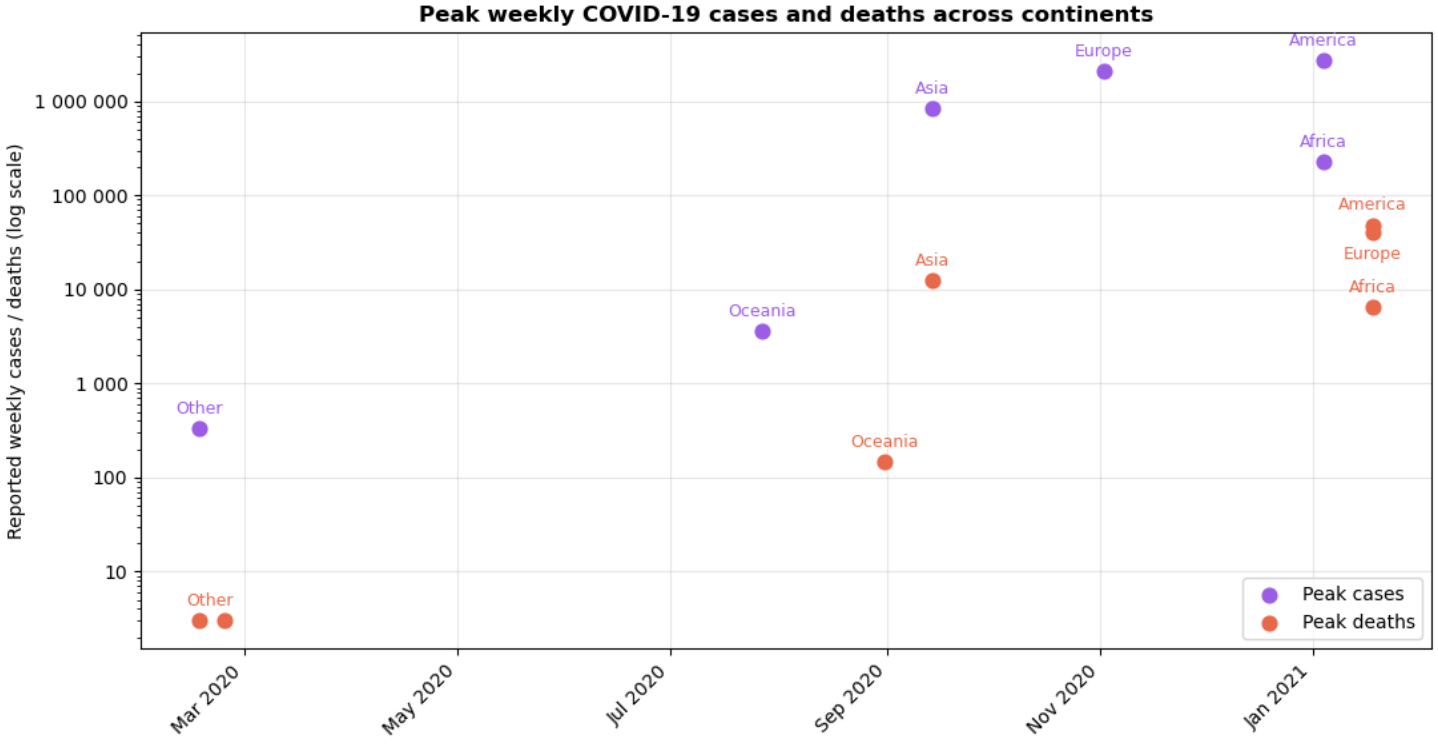


Figure 10. Weekly peaks of COVID-19 cases and deaths across continents.
The scatter plot compares the weekly peaks of reported cases and deaths for each continent over the study period.

CONCLUSION

This project demonstrated the use of Hadoop, MongoDB, and PySpark for processing and analysing epidemiological data. The three frameworks produced identical outputs across the selected MapReduce operations, confirming the consistency of their analytical results.

The analytical studies provided several insights into the COVID-19 dataset. They explored the relationship between reported cases and deaths, the influence of population size, differences in deaths-to-cases ratios across countries, geographical variations in cumulative cases and deaths, and the timing of epidemic peaks across continents. Specific MapReduce approaches with different frameworks were meant to be used to process huge datasets and extract meaningful information.

However, their execution times and resource consumption differed considerably. MongoDB recorded the shortest execution time, followed by PySpark and Hadoop. These measurements include framework-specific overheads and are based on four runs per framework. In particular, Hadoop's execution time includes job initialization and input file access, whereas MongoDB processes data already imported into the database. CPU and memory monitoring also revealed different resource-consumption patterns, with PySpark and Hadoop showing higher CPU usage during execution. Also, the choice of the machine heavily impacts the benchmark.

Overall, the results highlight that framework performance depends on several factors, including initialization overhead, data access, resource consumption, and the execution environment. Although MongoDB achieved the shortest execution time in this benchmark, these results cannot be directly generalised to larger datasets or more complex workloads. The choice of framework therefore depends on the size of the dataset, the complexity of the analysis, and the available infrastructure.

REFERENCES

Agoulmine, N. (n.d.). *Introduction to cloud computing and big data* [Course materials].

Apache Software Foundation. (n.d.). *Hadoop streaming*. Hadoop 2.7.0 documentation. https://hadoop.apache.org/docs/r2.7.0/hadoop-streaming/HadoopStreaming.html

Docker. (n.d.). *docker container stats*. https://docs.docker.com/reference/cli/docker/container/stats/

Humanitarian Data Exchange. (n.d.). *ECDC COVID-19 data*. https://data.humdata.org/dataset/ecdc-covid-19

Matplotlib. (n.d.). *matplotlib.pyplot.figure*. https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.figure.html

OpenClassrooms. (n.d.). *Allez au-delà de MapReduce avec Spark*. https://openclassrooms.com/fr/courses/4297166-realisez-des-calculs-distribues-sur-des-donnees-massives/4308661-allez-au-dela-de-mapreduce-avec-spark

Shivogojohn. (n.d.). *Pearson correlation coefficient in Python*. Medium. https://medium.com/@shivogojohn/pearson-correlation-coefficient-in-python-74fb569f2b7f

Spark Summit. (2014). *Spark tutorials*. http://spark-summit.org/2014/training

The Graph Gallery. (n.d.). *The ggplot2 package*. https://r-graph-gallery.com/ggplot2-package.html

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*, 15–28. https://www.cs.berkeley.edu/~matei/papers/2012/nsdi_spark.pdf